

Lecture 6: ECE C147A/C247A, Neural Networks & Deep Learning

Neural Networks, back propagation and optimization
October 14th, 2026

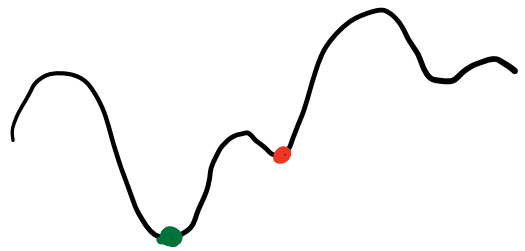
These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

Announcements

- HW #2 is due Friday October 16, 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late**
- HW#1 solutions posted.
- **Please read the rules handout carefully including for late HWs etc.**
- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.
- Discussion sessions held on Fridays 11am, will be recorded and live zoom link also is available.
- For midterm **CAE approved accommodation students have to contact CAE office to proctor exam** as we cannot do this ourselves. Please do this pro-actively.

Review of previous lecture

- **Softmax based classifier**
  - $\text{softmax}_i(\mathbf{x}) = \dfrac{e^{a_i(\mathbf{x})}}{\sum_{j=1}^{c} e^{a_j(\mathbf{x})}}$ where $a_i(\mathbf{x}) = \mathbf{w}_i^\top \mathbf{x} + b_i$, and we interpret
  - $\text{Prob}(y^{(i)} = j \,|\, \mathbf{x}^{(i)}, \theta) = \text{softmax}_j(\mathbf{x}^{(i)})$
  - Numerical issues, overflow and normalization
  - Loss function: $J(\boldsymbol{\theta}) = -\log \mathscr{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{m} \left(\log \sum_{j=1}^{c} \left\{ e^{a_j(\mathbf{x}^{(i)})} \right\} - a_{y^{(i)}}(\mathbf{x}^{(i)}) \right)$
  - find appropriate $\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
- **Question:** how to solve for this?
- **Approach:** Gradient descent
  - $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$, where η is the learning rate (step-size) and the gradient of the loss function $J(\boldsymbol{\theta})$.
  - Need to choose the learning rate appropriately.
  - $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} \left(a_{y^{(i)}}(\mathbf{x}^{(i)}) - \log \sum_{j=1}^{c} \left\{ e^{a_j(\mathbf{x}^{(i)})} \right\} \right)$



Review of previous lecture (Contd.)

- Full batch, versus minibatch stochastic gradient descent

seen in HW1

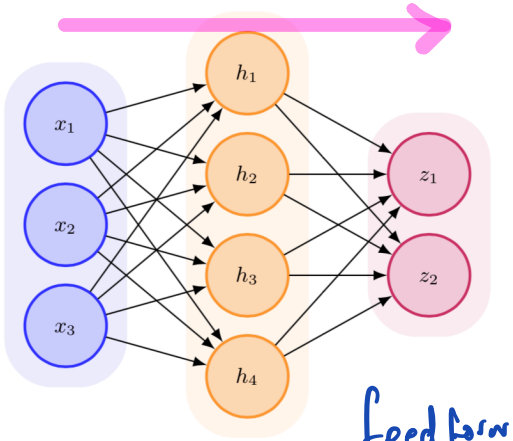
use k out of m random samples to compute stochastic gradient

k=1 typically called SGD

- Connect biological neuronal models and artificial neural networks

$$\mathbf{h}_1 = f\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right)$$
$$\mathbf{h}_2 = f\left(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2\right)$$
$$\mathbf{z} = f\left(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3\right)$$

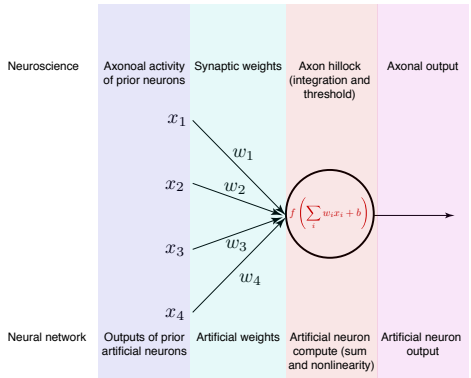
non-linearity



Multilayer perceptron

↳ could have many hidden layers

feed forward neural network



→ inspiration for NN

What if f() is linear?

Neural networks

$f(\underline{x}) = \underline{x}$ → identity function

The above figure suggests the following equation for a neural network.

- Layer 1: $\mathbf{h}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$ ←
- Layer 2: $\mathbf{h}_2 = \mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2$
- ⋮
- Layer N: $\mathbf{z} = \mathbf{W}_N\mathbf{h}_{N-1} + \mathbf{b}_N$

$\underline{h}_2 = \underline{\underline{W}}_2 \underline{h}_1 + \underline{b}_2 = \underline{\underline{W}}_2 [\underline{\underline{W}}_1 \underline{x} + \underline{b}_1] + \underline{b}_2$

$= \underbrace{\underline{\underline{W}}_2 \underline{\underline{W}}_1}_{\tilde{\underline{\underline{W}}}_2} \underline{x} + \underbrace{\underline{\underline{W}}_2 \underline{b}_1 + \underline{b}_2}_{\tilde{\underline{b}}_2} = \tilde{\underline{\underline{W}}}_2 \underline{x} + \tilde{\underline{b}}_2$

Any composition of linear functions can be reduced to a single linear function. Here, $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where

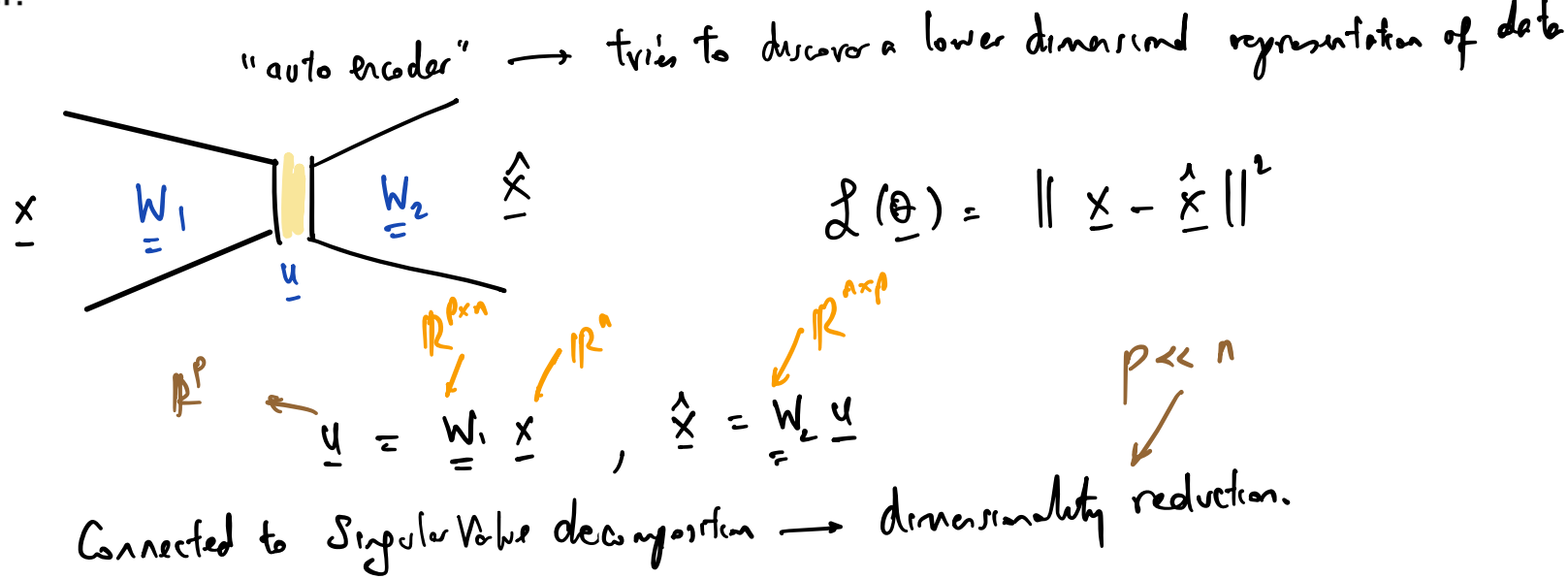
$$\mathbf{W} = \mathbf{W}_N \cdots \mathbf{W}_2\mathbf{W}_1$$

and

$$\mathbf{b} = \mathbf{b}_N + \mathbf{W}_N\mathbf{b}_{N-1} + \cdots + \mathbf{W}_N \cdots \mathbf{W}_3\mathbf{b}_2 + \mathbf{W}_N \cdots \mathbf{W}_2\mathbf{b}_1$$

What if f() is linear?

- This may be useful in some contexts. For example, when $\dim(\mathbf{h}) \ll \dim(\mathbf{x})$, this corresponds to finding a low-rank representation of the inputs.
- However, a system with greater complexity may require a higher capacity model.



Introducing nonlinearity

To increase the network capacity, we can make it nonlinear. We do this by introducing a nonlinearity, $f(\cdot)$, at the output of each artificial neuron.

non-linear

- Layer 1: $\mathbf{h}_1 = f(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$
- Layer 2: $\mathbf{h}_2 = f(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$
- $\vdots$
- Layer N: $\mathbf{z} = \mathbf{W}_N\mathbf{h}_{N-1} + \mathbf{b}_N$

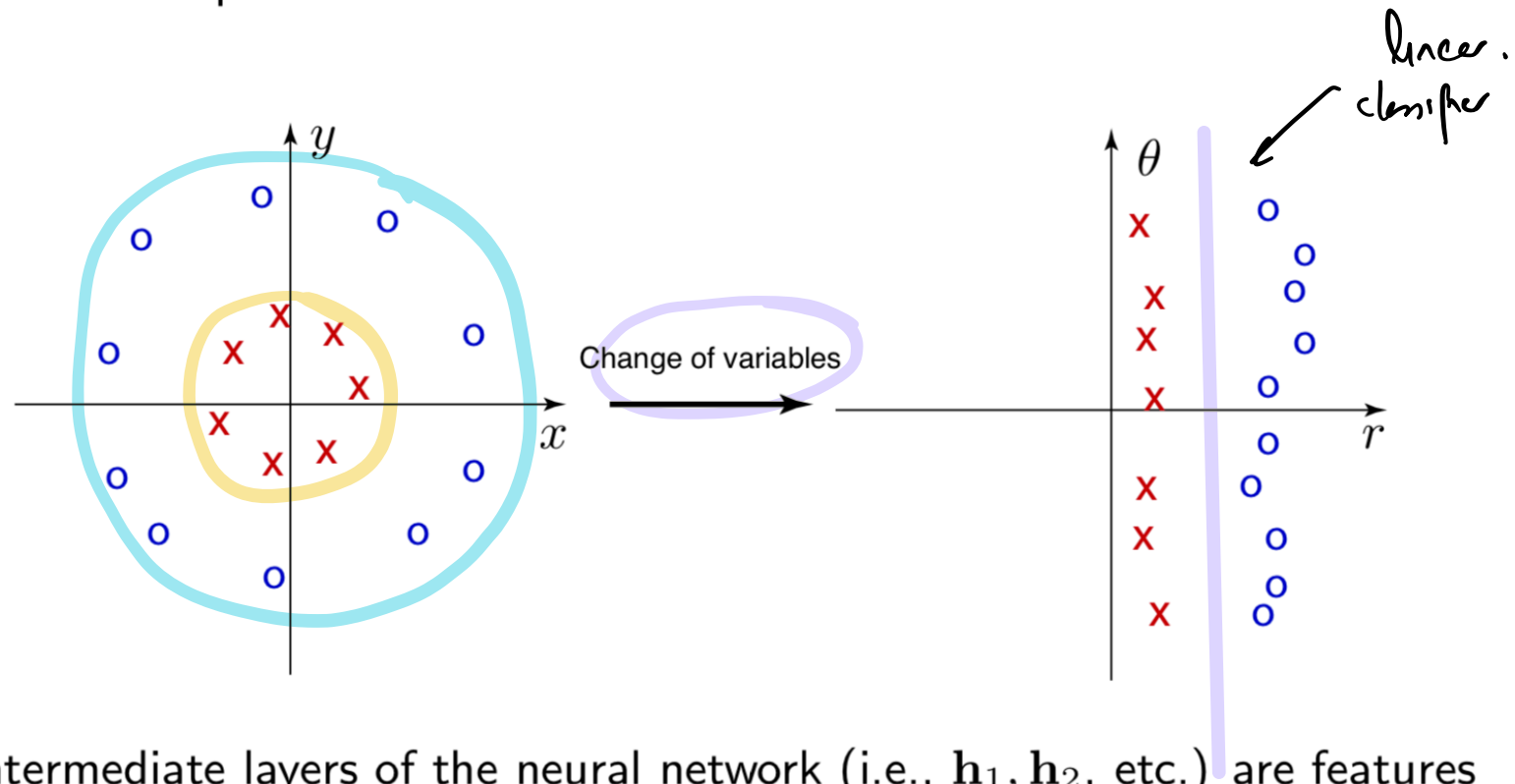
Intuition: Non-linear layers "convert" input to a "simpler" representation to which we can apply a linear classifier (last layer)

A few notes:

- These equations describe a *feedforward neural network*, also called a *multilayer perceptron*.
- $f(\cdot)$ is typically called an *activation function* and is applied elementwise on its input.
- The activation function does not typically act on the output layer, $\mathbf{z}$, as these are meant to be interpreted as scores. Instead, separate "output activations" are used to process $\mathbf{z}$. While these output activations may be the same as the activation function, they are typically different. For example, it may comprise a softmax or SVM classifier.

A perspective on feature learning

One area of machine learning is very interested in finding *features* of the data that are then good for use as the input data to a classifier (like a SVM). Why might this be important?



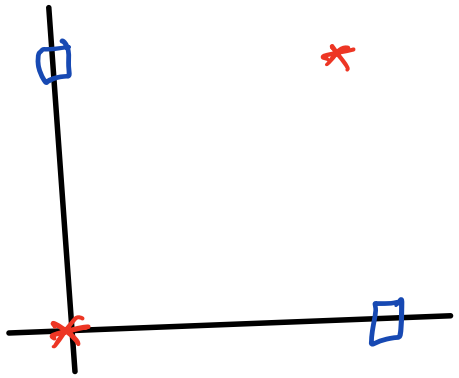
The intermediate layers of the neural network (i.e., $\mathbf{h}_1, \mathbf{h}_2$, etc.) are features that the later layers then use for decoding. If the performance of the neural network is well, these features are good features.

Importantly, these features don't have to be handcrafted.

Example: XOR

Consider a system that produces training data that follows the $\mathrm{xor}(\cdot)$ function. The xor function accepts a 2-dimensional vector $\mathbf{x}$ with components x_1 and x_2 and returns 1 if $x_1 \neq x_2$. Concretely,

x_1	x_2	$\mathrm{xor}(\mathbf{x})$
0	0	0
0	1	1
1	0	1
1	1	0



$$J(\theta) = \frac{1}{2}\sum_{\mathbf{x}}(g(\mathbf{x}) - y(\mathbf{x}))^2$$

(Note, we wouldn't know $\mathrm{xor}(\mathbf{x})$, but we would have samples of corresponding inputs and outputs from training data. Hence, it may be better to simply replace $\mathrm{xor}(\mathbf{x})$ with $y(\mathbf{x})$ representing training examples.)

Example: XOR

Consider first a linear approximation of xor, via $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$. Then,

$$\frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}} = \sum_{\mathbf{x}} (\mathbf{w}^T\mathbf{x} + b - y(\mathbf{x}))\mathbf{x}$$

$$\frac{\partial J(\mathbf{w}, b)}{\partial b} = \sum_{\mathbf{x}} (\mathbf{w}^T\mathbf{x} + b - y(\mathbf{x}))$$

Equating these to 0, we arrive at:

$$(w_1 + b - 1)\begin{bmatrix} 1 \\ 0 \end{bmatrix} + (w_2 + b - 1)\begin{bmatrix} 0 \\ 1 \end{bmatrix} + (w_1 + w_2 + b)\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

These two equations can be simplified as:

$$\begin{aligned} (w_1 + b - 1) + (w_1 + w_2 + b) &= 0 \\ (w_2 + b - 1) + (w_1 + w_2 + b) &= 0 \end{aligned}$$

These equations are symmetric, implying $w_1 = w_2 = w$. This means:

$$3w + 2b - 1 = 0 \implies b = \frac{1 - 3w}{2}$$



Example: XOR

Now let's consider using a two-layer neural network, with the following equation:

$$g(\mathbf{x}) = \mathbf{w}^T \max(0, \mathbf{W}^T \mathbf{x} + \mathbf{c}) + b$$

We haven't yet discussed how to optimize these parameters, but the point here is to show that by introducing a simple nonlinearity like $f(x) = \max(0, x)$, we can now solve the $\mathrm{xor}(\cdot)$ problem. Consider the solution:

$$\begin{aligned} \mathbf{W} &= \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \\ \mathbf{c} &= [0, -1]^T \\ \mathbf{w} &= [1, -2]^T \end{aligned}$$



What nonlinearity to use?

There are a variety of activation functions. We'll discuss some more commonly encountered ones.

Potential guiding principle:

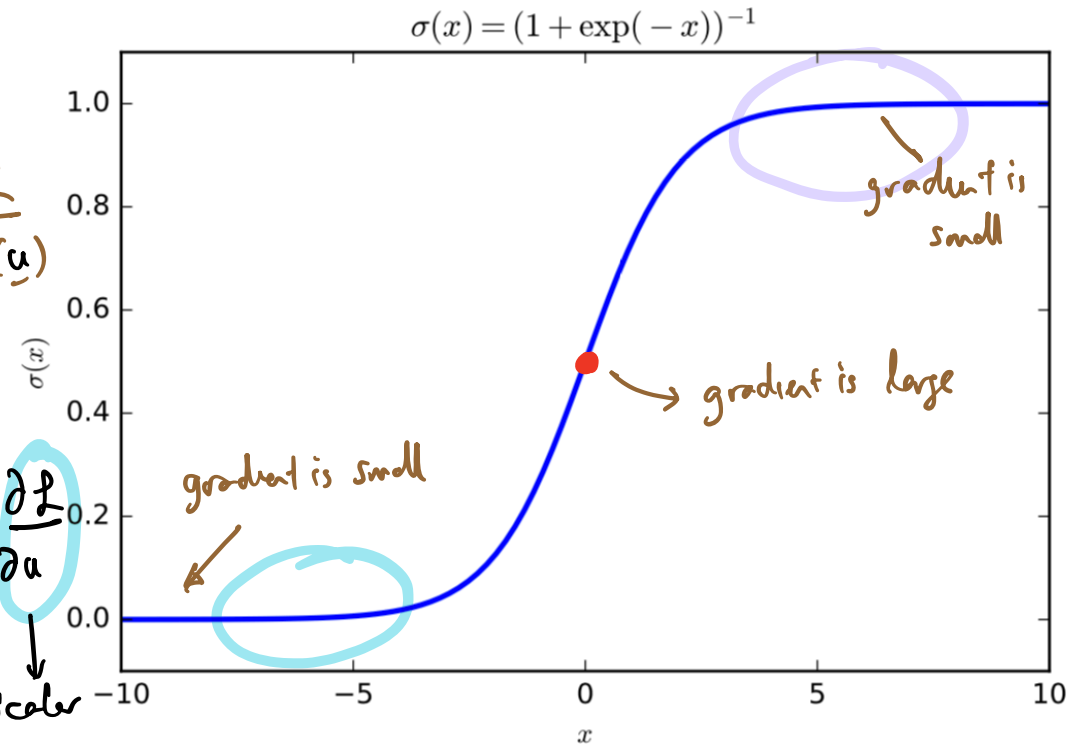
• design of loss functn coupled to non-linearity

"One recurring theme throughout neural network design is that the gradient of the cost function must be large and predictable enough to serve as a good guide for the learning algorithm." (Goodfellow et al., p. 173)

Sigmoid unit

Sigmoid activation, $\sigma(x) = \frac{1}{1+\exp(-x)}$

```
f = lambda x: 1.0 / (1.0 + np.exp(-x))
```



$\frac{\partial L}{\partial W} = \frac{\partial \sigma(u)}{\partial W} \cdot \frac{\partial L}{\partial \sigma(u)}$ ← chain rule

$u = W^T x + b$

$= \underbrace{\sigma(u)\cdot[1-\sigma(u)]}_{\geq 0} \cdot x \frac{\partial L}{\partial u}$ ← scalar

$\sigma(x) \in [0,1]$

$\Rightarrow 1-\sigma(x) \in [0,1]$

Its derivative is:

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1-\sigma(x))$$ → do this as an exercise

Sigmoid unit

Sigmoid activation, $\sigma(x) = \frac{1}{1+\exp(-x)}$

Pros:

- Around $x = 0$, the unit behaves linearly.
- It is differentiable everywhere.

Cons:

- At extremes, the unit *saturates* and thus has zero gradient. This results in no learning with gradient descent.
- The sigmoid unit is not zero-centered; rather its outputs are centered at 0.5.

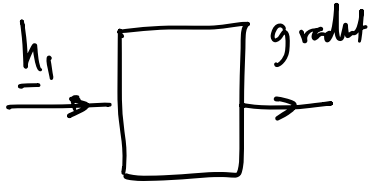
Consider $f(\mathbf{w}) = \sigma(\mathbf{w}^T\mathbf{x} + b)$. The derivative with respect to $\mathbf{w}$, the parameters, is:

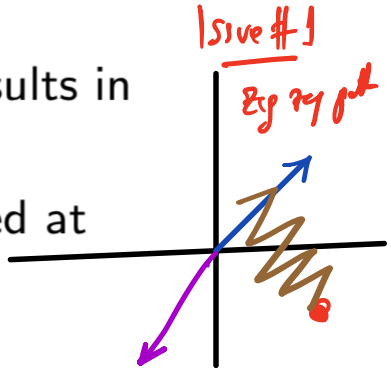
$$\frac{\partial f(\mathbf{w})}{\partial \mathbf{w}} = \sigma(z)(1-\sigma(z))\mathbf{x}$$

If $\mathbf{x} \geq 0$ (e.g., if the prior unit had a sigmoidal output), then the gradient has all positive entries or all negative entries.

This can result in zig-zagging during gradient descent.

- As a result, the sigmoid activation is rarely used.



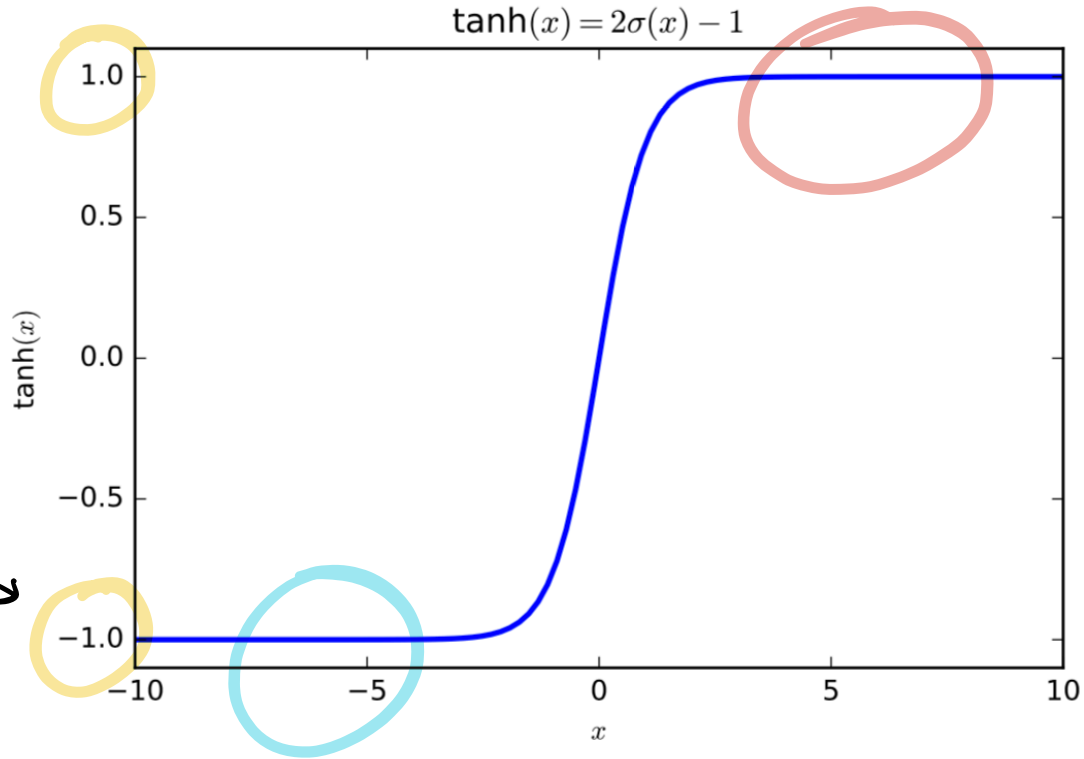


Hyperbolic tangent

Hyperbolic tangent, $\tanh(x) = 2\sigma(x) - 1$

The hyperbolic tangent is a zero-centered sigmoid-looking activation.

```
f = lambda x: np.tanh(x)
```



remove the zig-zag issue since the value of the h_i could be +ve or -ve

Its derivative is:

$$\frac{d\tanh(x)}{dx} = 1 - \tanh^2(x)$$



Hyperbolic tangent

Hyperbolic tangent, $\tanh(x) = 2\sigma(x) - 1$

Pros:

- Around $x = 0$, the unit behaves linearly.
- It is differentiable everywhere.
- It is zero-centered, fixing a major flaw of the sigmoid activation.

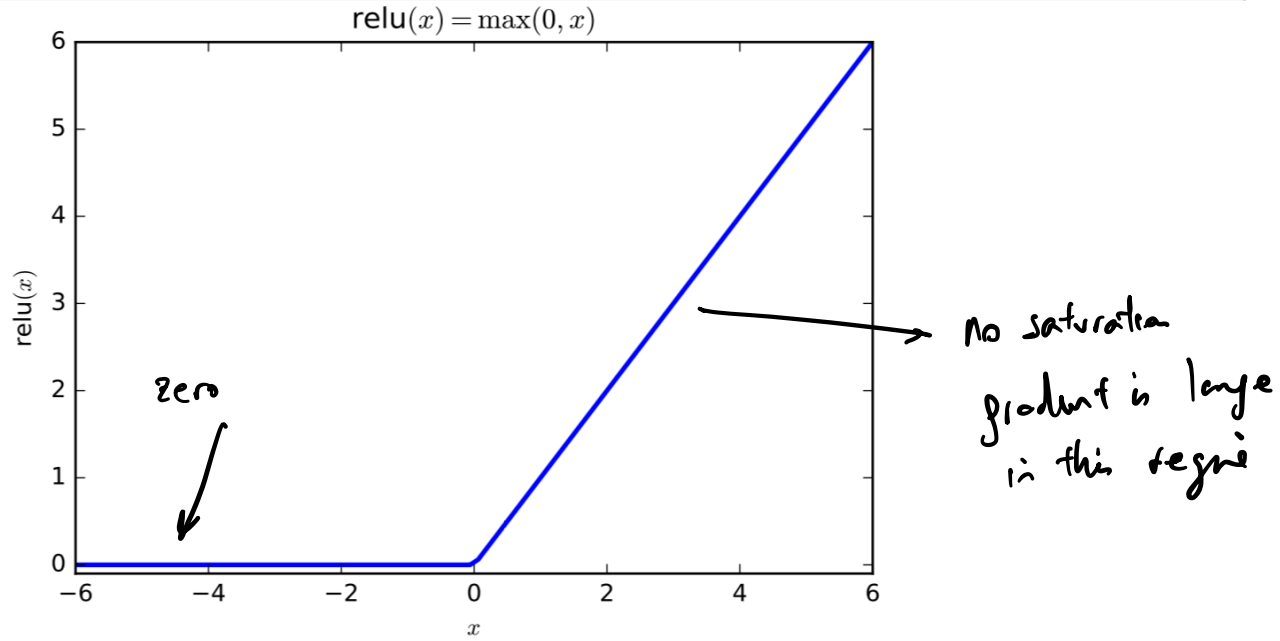
Cons:

- Like the sigmoid unit, when a unit saturates, i.e., its values grow larger or smaller, the unit saturates and no additional learning occurs.
- The gradient is subject to second order effects; as $|x|$ grows, the gradient decreases to zero. This makes the gradient smaller at larger $|x|$, which may distort the gradient.

ReLU unit

Rectified linear unit, $\mathbf{ReLU}(x) = \max(0, x)$

```
f = lambda x: x * (x > 0)
```



Its derivative is:

$$\frac{d\mathrm{ReLU}(x)}{dx} = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

This function is not differentiable at $x = 0$. However, we can define its subgradient by setting the derivative to be between $[0, 1]$ at $x = 0$.



ReLU unit

Rectified linear unit, $\mathbf{ReLU}(x) = \max(0, x)$

Pros:

- In practice, learning with the ReLU unit converges faster than sigmoid and tanh. AlexNet winner of ImageNet competition in 2012 used ReLU → empirically the training took 6x less than tanh
- When a unit is active, it behaves as a linear unit.
- The derivative at all points, except $x = 0$, is 0 or 1. When $x > 0$, the gradients are large, and not scaled by second order effects.
- There is no saturation if $x > 0$.

Cons:

relu all +ve values → zig zag issue

- $\mathrm{ReLU}(x)$, like sigmoid, is not zero-centered.
- $\mathrm{ReLU}(x)$ is not differentiable at $x = 0$. However, in practice, this is not a large issue. A heuristic when evaluating $\frac{d\mathrm{ReLU}(x)}{dx}\Big|_{x=0}$ is to return the left derivative (0) or the right derivative (1); this is reasonable given digital computation is subject to numerical error.
- Learning does not happen for examples that have zero activation. This can be fixed by e.g., using a leaky ReLU or maxout unit.

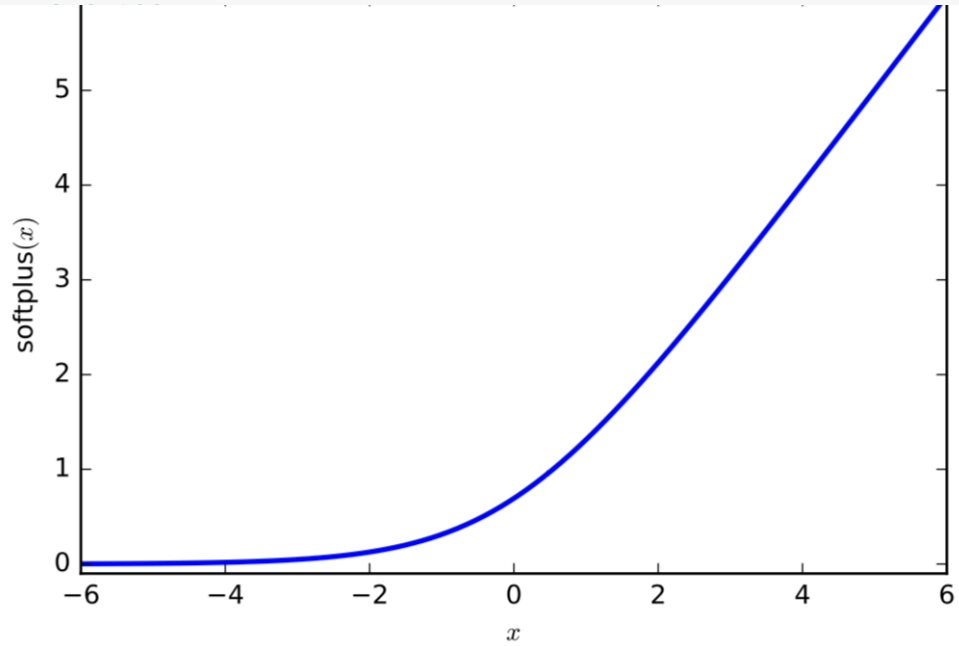
Softplus unit

Softplus unit, $\zeta(x) = \log(1 + \exp(x))$

for large x $\zeta(x) \approx x$

One may consider using the softplus function, $\zeta(x) = \log(1 + e^x)$, in place of $\mathrm{ReLU}(x)$. Intuitively, this ought to work well as it resembles $\mathrm{ReLU}(x)$ and is differentiable everywhere. However, empirically, it performs worse than $\mathrm{ReLU}(x)$.

```
f = lambda x: np.log(1+np.exp(x))
```



Its derivative is:

$$\frac{d\zeta(x)}{dx} = \sigma(x)$$

Softplus unit

"One might expect it to have an advantage over the [ReLU] due to being differentiable everywhere or due to saturating less completely, but empirically it does not." (Goodfellow et al., p. 191)

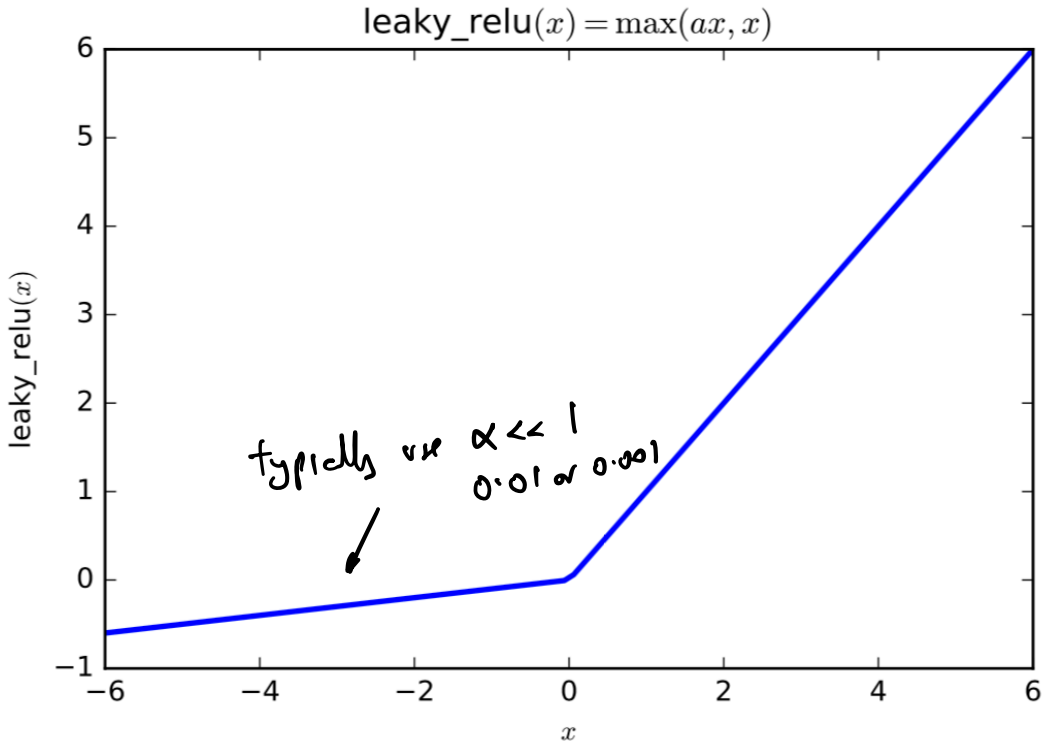
Neuron	MNIST	CIFAR10	NISTP	NORB
***With* unsupervised pre-training**				
Rectifier	**1.20%**	**49.96%**	**32.86%**	**16.46%**
Tanh	**1.16%**	**50.79%**	35.89%	17.66%
Softplus	**1.17%**	**49.52%**	**33.27%**	19.19%
***Without* unsupervised pre-training**				
Rectifier	**1.43%**	**50.86%**	**32.64%**	**16.40%**
Tanh	1.57%	52.62%	36.46%	19.29%
Softplus	1.77%	53.20%	35.48%	17.68%

Glorot et al., 2011a

Leaky ReLU / PReLU unit

Leaky rectified linear unit, $f(x) = \max(\alpha x, x)$

```
f = lambda x: x * (x>0) + 0.1*x * (x<0)
```

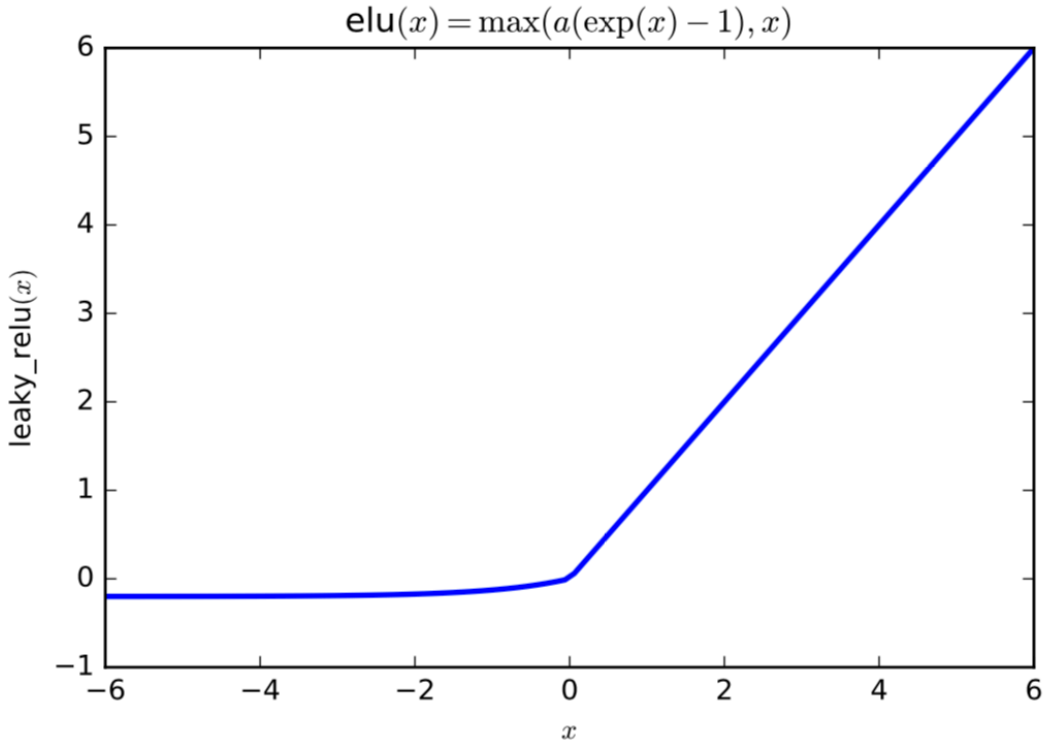


The leaky ReLU avoids the stopping of learning when $x < 0$. α may be treated as a selected hyperparameter, or it may be a parameter to be optimized in learning, in which case it is typically called the "PReLU" for parametrized rectified linear unit.

ELU unit

Exponential linear unit, $f(x) = \max(\alpha(\exp(x) - 1), x)$

```
f = lambda x: x * (x>0) + 0.2*(np.exp(x) - 1) * (x<0)
```

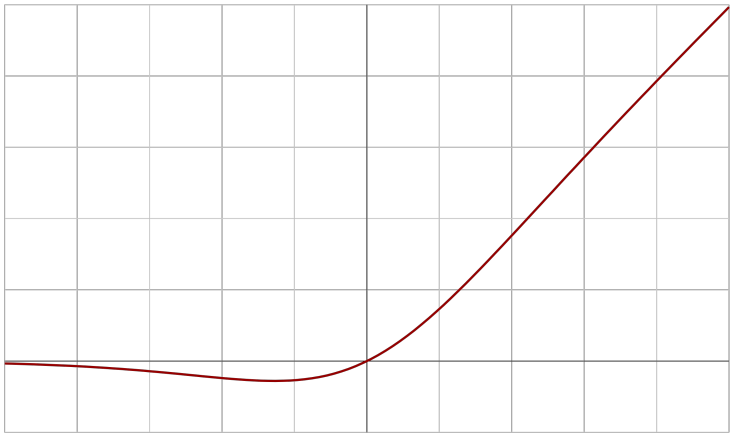


The exponential linear unit avoids the stopping of learning when $x < 0$. A con of this activation function is that it requires computation of the exponential, which is more expensive.

More recently…

used in diffusion models

$$\text{swish}(x) = x\,\text{sigmoid}(\beta x) = \frac{x}{1+e^{-\beta x}}.$$



potentially GPT

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}(x/\sqrt{2})\right]$$

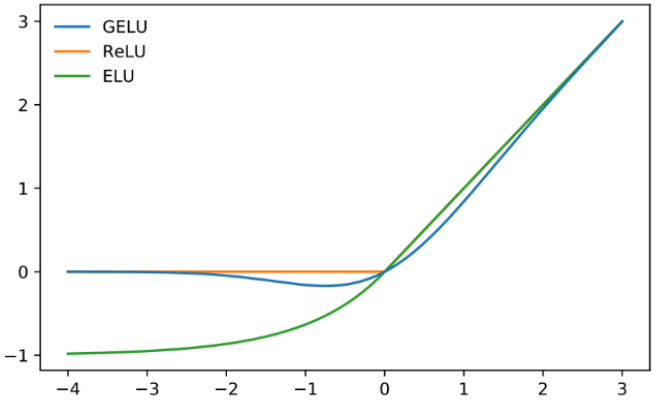
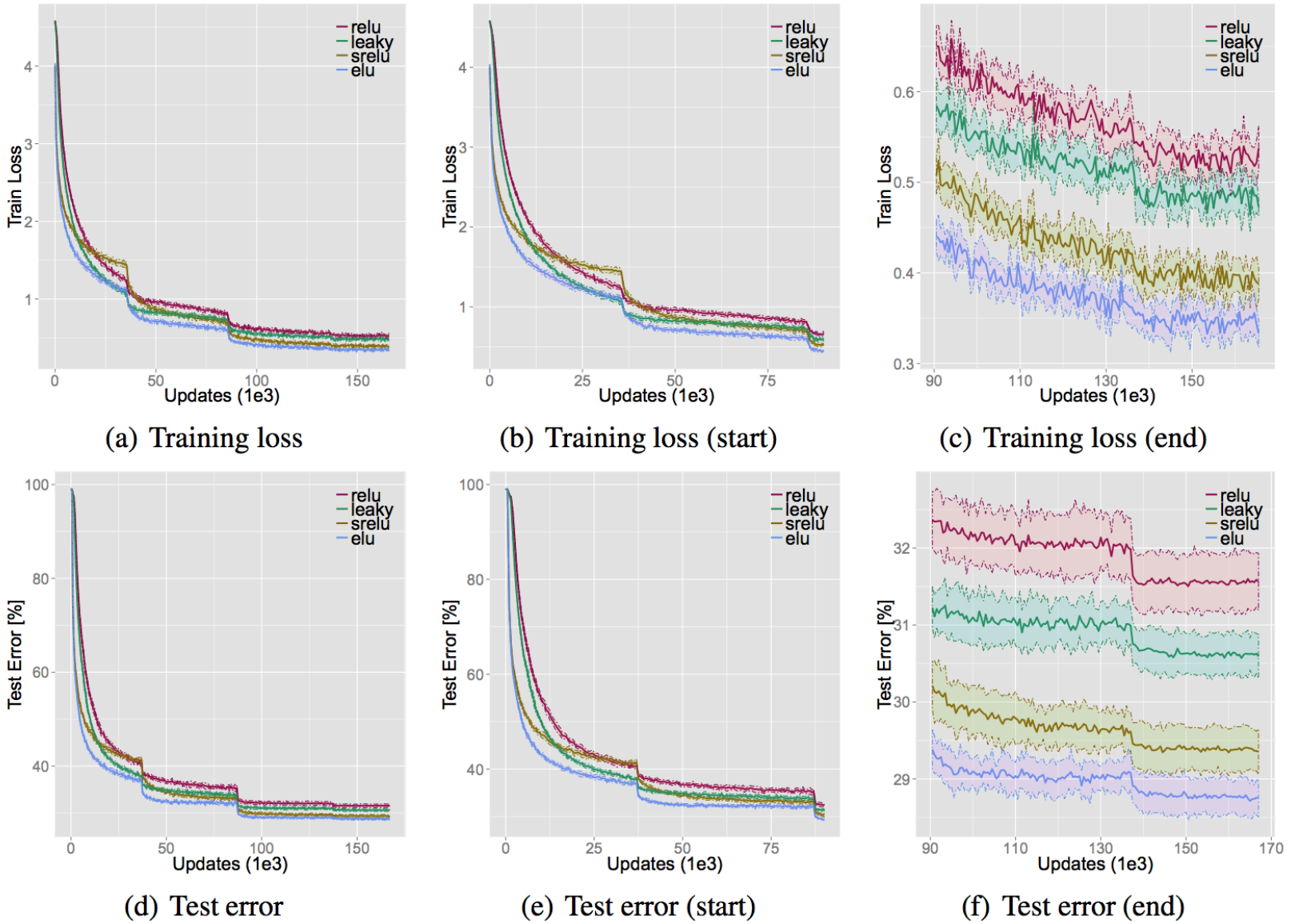


Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

Which LU do I use?



(a) Training loss (b) Training loss (start) (c) Training loss (end)

(d) Test error (e) Test error (start) (f) Test error (end)

Figure 4: Comparison of ReLUs, LReLUs, and SReLUs on CIFAR-100. Panels (a-c) show the

Clevert et al., 2015



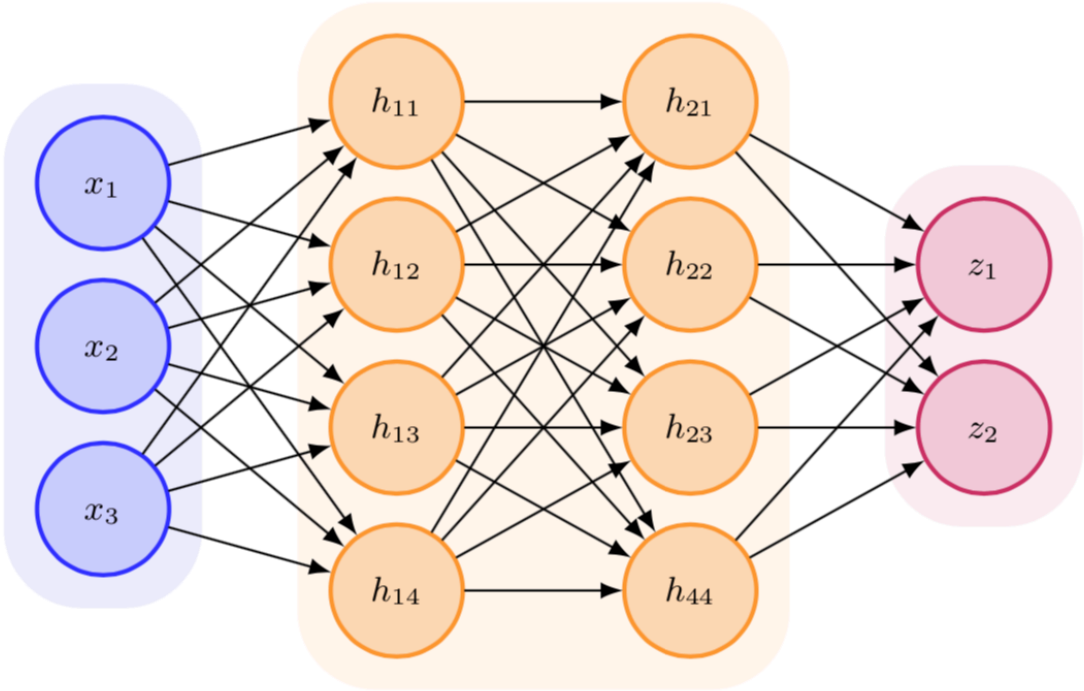
What activation function do I use?

In practice...

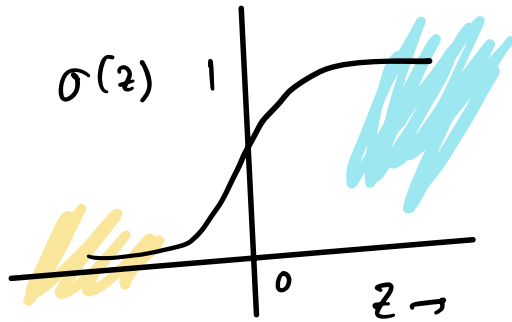
In practice...

- The ReLU unit is very popular.
- The sigmoid unit is almost never used; tanh is preferred.
- It may be worth trying out leaky ReLU / PReLU / ELU / maxout for your application.
- This is an active area of research.

Back to NN architecture



- Layer 1: $\mathbf{h}_1 = f(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$
- Layer 2: $\mathbf{h}_2 = f(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$
- $\vdots$
- Layer N: $\mathbf{z} = \mathbf{W}_N\mathbf{h}_{N-1} + \mathbf{b}_N$



What output activation do we use?

The output activation interacts with the cost function

$\sigma(z) \in [0,1]$ → $\Pr\{\text{image } x^{(i)} \text{ is in class } 1\}$

CLASSIFICATION: $y^{(i)} \in \{0,1\}$ — class 0, class 1

$z = w^T x + b$ when we apply a linear classifier

when z is large → class 1, z is large +ve

z is large −ve → class 0

What outputs and cost functions?

There are several options to process the output scores, $\mathbf{z}$, to ultimately arrive at a cost function. The choice of output units interacts with what cost function to use.

Example: Consider a neural network that produces a single score, z, for binary classification. As the output unit, we choose the sigmoid nonlinearity, so that $\hat{y} = \sigma(z)$. On a given example, $y^{(i)}$ is either 0 or 1, and $\hat{y}^{(i)} = \sigma(z^{(i)})$ can be interpreted as the algorithm's probability the output is in class 1. Is it better to use mean-square error or cross-entropy (i.e., corresponding to maximum-likelihood estimation) as the cost function? For n examples:

$$\text{MSE} = \frac{1}{2}\sum_{i=1}^{n}\left(y^{(i)} - \sigma(z^{(i)})\right)^2$$

$$\text{CE} = -\sum_{i=1}^{n}\left[y^{(i)}\log\sigma(z^{(i)}) + (1-y^{(i)})\log(1-\sigma(z^{(i)}))\right]$$

Cross-entropy

two different loss functions

A picture for intuition

MSE loss function $\hat{y}^{(i)} = \sigma(\underbrace{w^T x^{(i)} + b}_{z^{(i)}})$

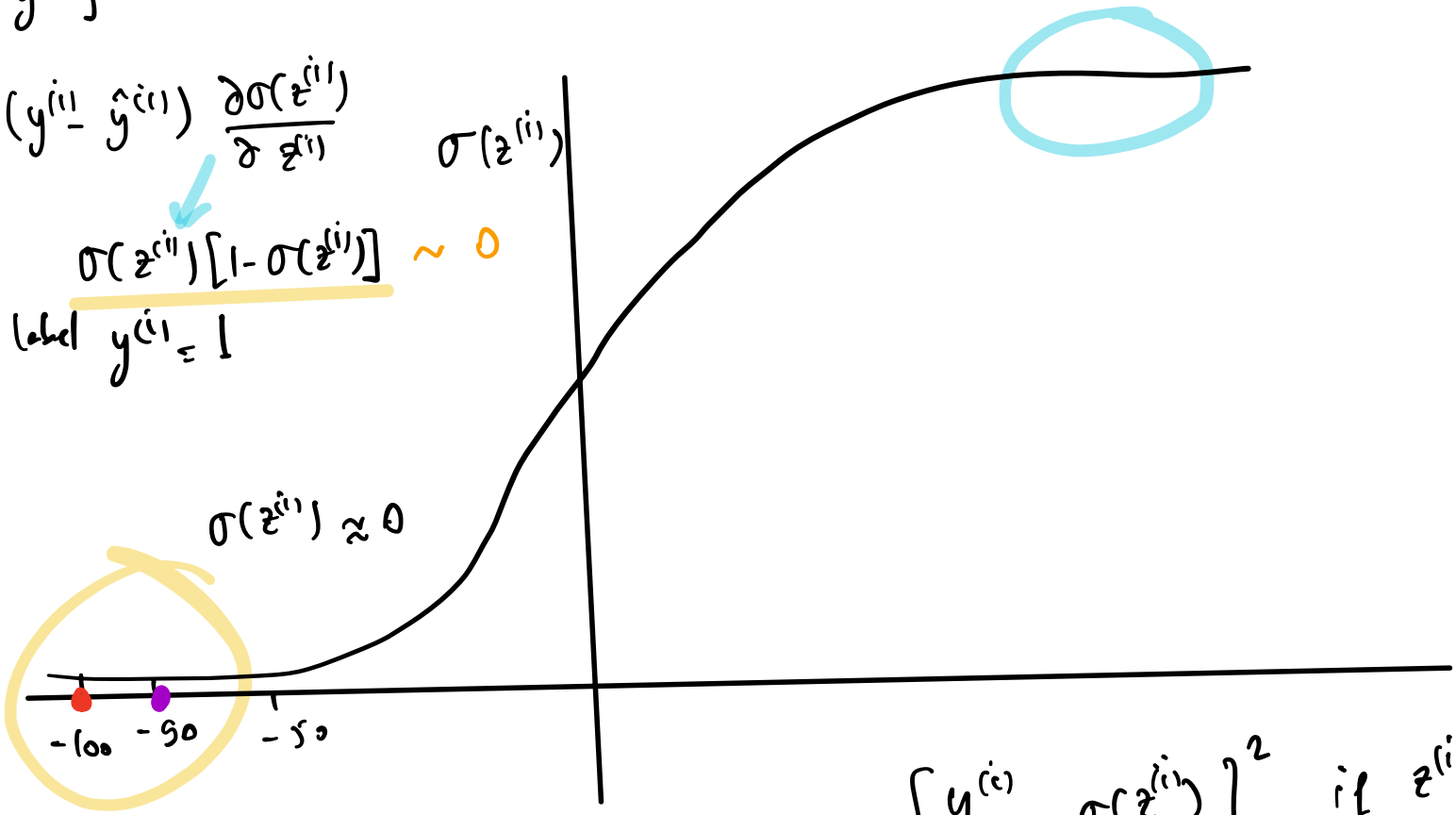
$z^{(i)} = w^T x + b$

$MSE_i = [y^{(i)} - \hat{y}^{(i)}]^2$

$\frac{\partial MSE_i}{\partial z^{(i)}} = 2(y^{(i)} - \hat{y}^{(i)}) \frac{\partial \sigma(z^{(i)})}{\partial z^{(i)}}$

$\sigma(z^{(i)})[1 - \sigma(z^{(i)})] \sim 0$

Let correct label $y^{(i)} = 1$



$[y^{(i)} - \sigma(z^{(i)})]^2$ if $z^{(i)} = -100$ or -50

$\sigma(-100) \approx 0 \approx \sigma(-50)$

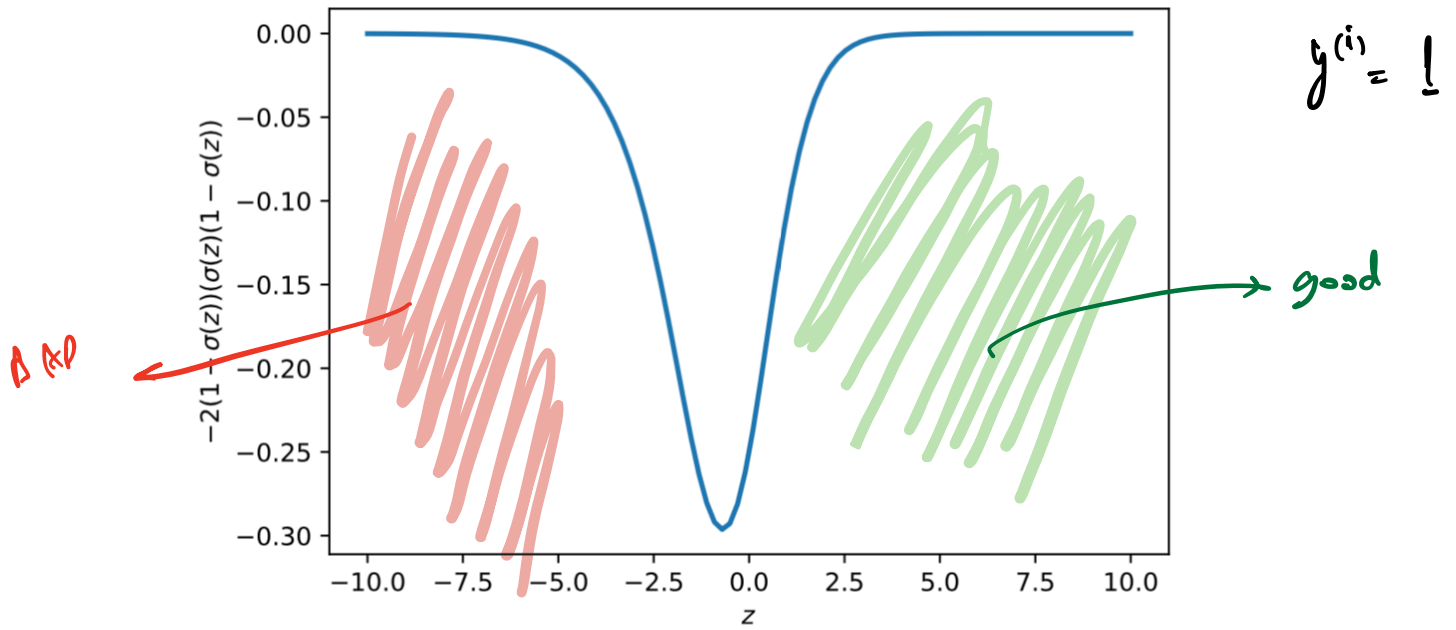
$(y^{(i)} - \sigma(-100))^2 \approx 1 \approx [y^{(i)} - \sigma(-50)]^2$

What outputs and cost functions? (cont.)

Example (cont): Consider just one example, where $y^{(i)} = 1$. For this example,

$$\frac{\partial \text{MSE}}{\partial z^{(i)}} = -2(y^{(i)} - \sigma(z^{(i)}))(\sigma(z^{(i)})(1 - \sigma(z^{(i)})))$$

This derivative looks like the following:

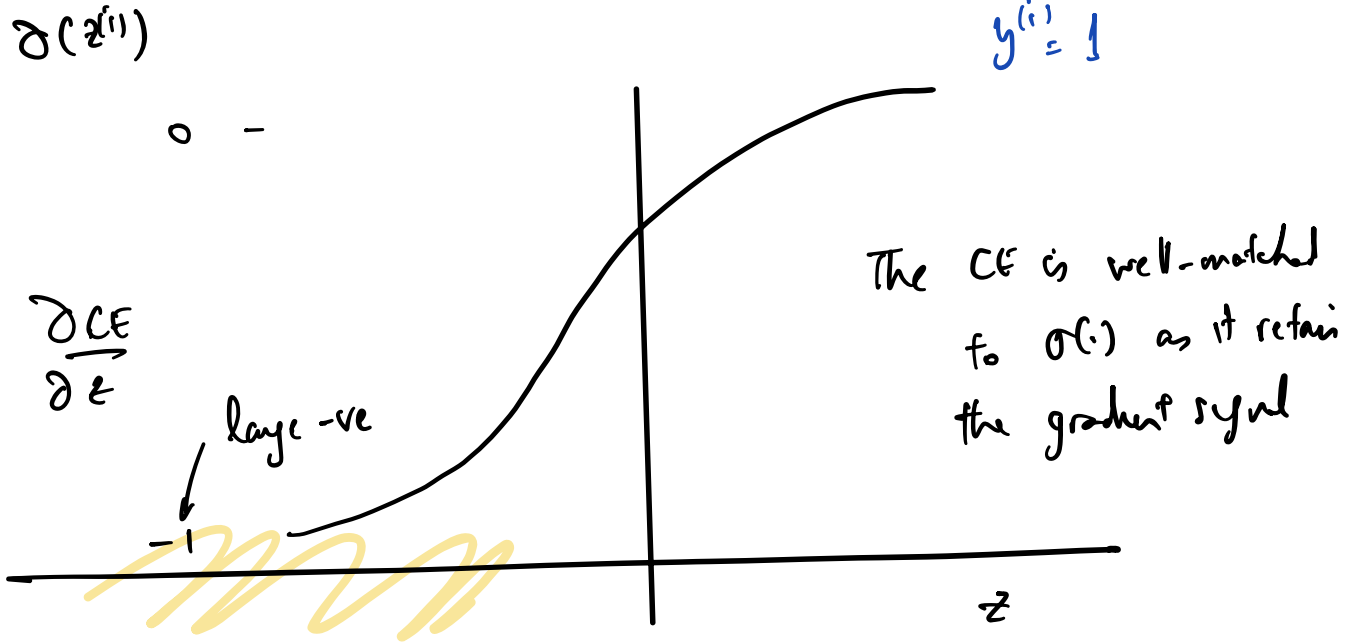


When z is very negative, indicating a large MSE, the gradient saturates to zero, and no learning occurs.

Review of previous lecture (Contd.)

Cross-entropy loss

$$\frac{\partial CE}{\partial (z^{(i)})} = \sigma(z^{(i)}) - 1$$



Bottom line Look at the gradient to jointly choose/design the matched loss function to non-linearity



The output activation interacts with the cost function

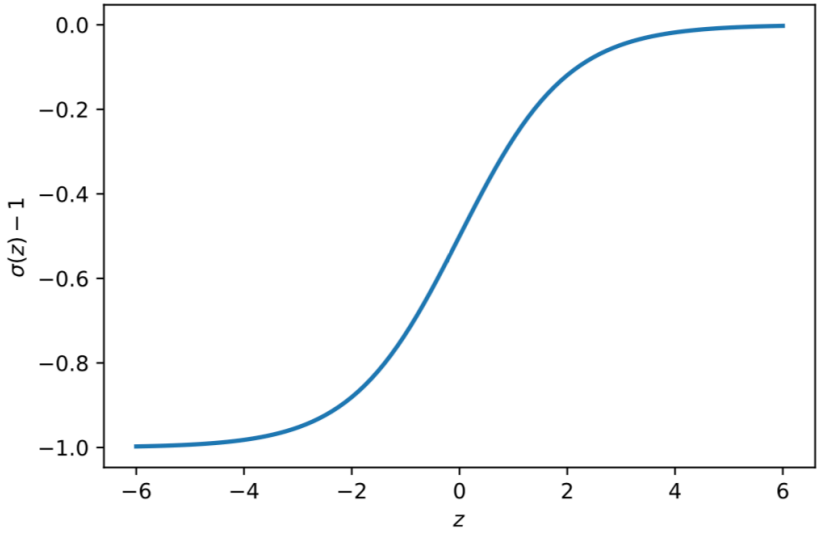
What outputs and cost functions? (cont.)

Example (cont): Now let's consider the cross-entropy. For one example, where $y^{(i)} = 1$,

$$\frac{\partial \text{CE}}{\partial z^{(i)}} = \sigma(z^{(i)}) - 1$$

This derivative looks like the following:

This is the more formal version of previous handwritten slide



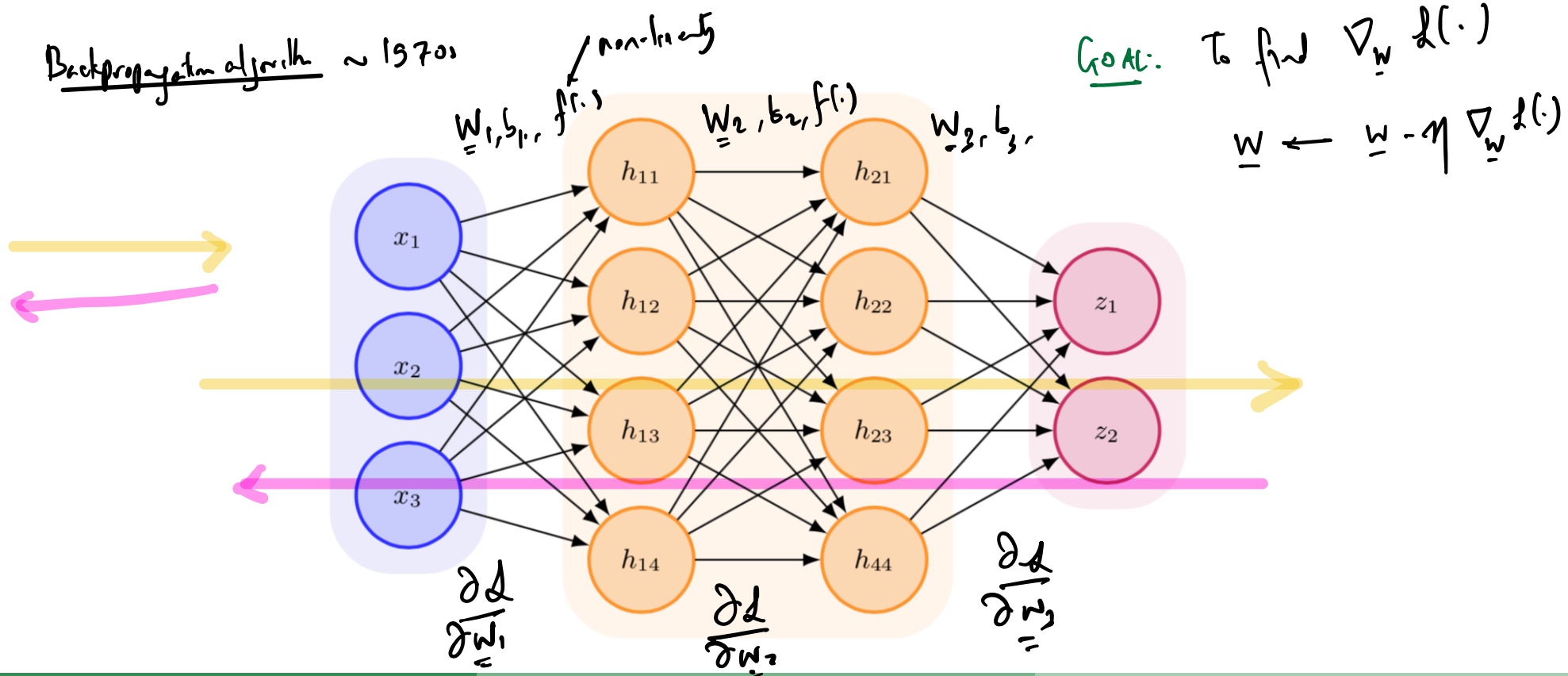
Notice that when z is very negative, learning will occur, and it will only "stall" when z gets close to the right answer.

What next?

Now that we've defined all the elements of the neural network, the question now becomes: how do we learn its parameters?

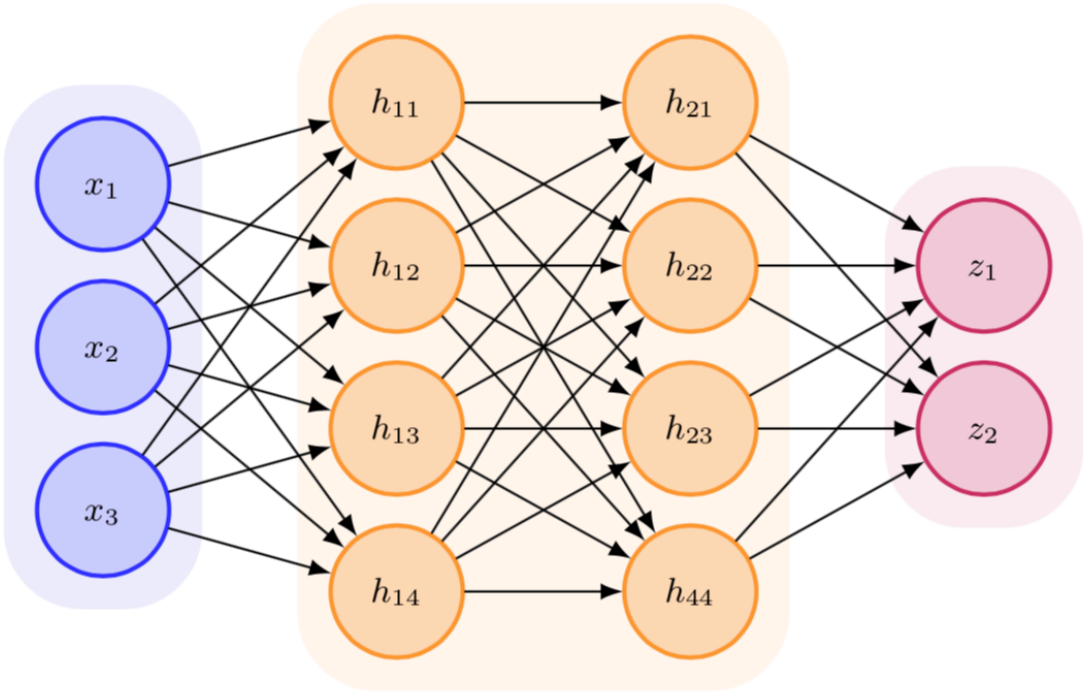
The short answer is that we use versions of gradient descent.

However, neural networks architectures have units that are several layers from the output. In these scenarios, how do we arrive at the gradient?



Backpropagation

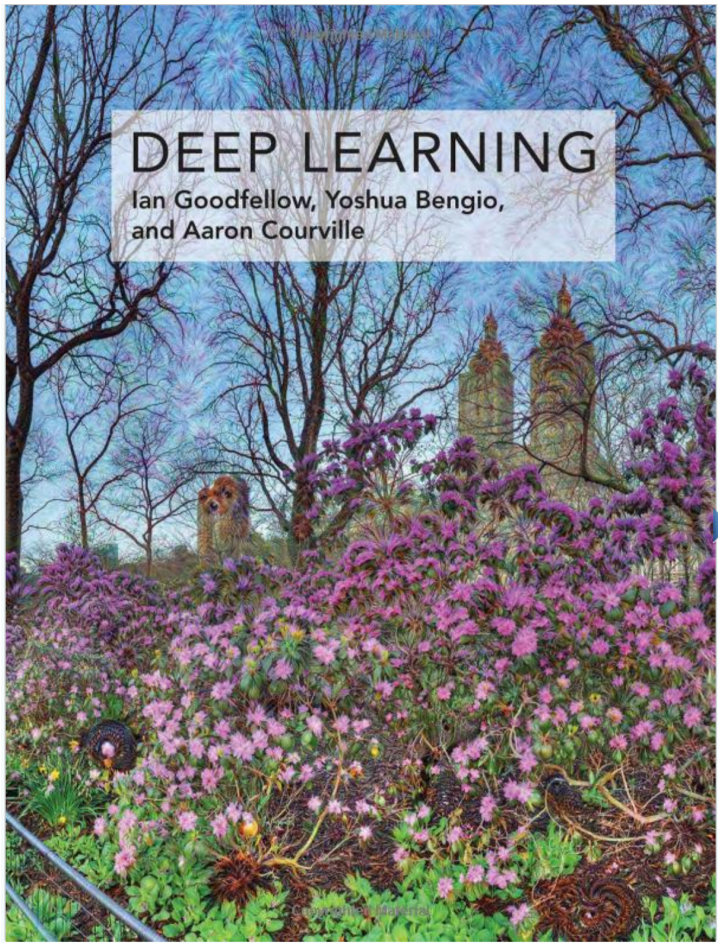
We introduce backpropagation as a technique to calculate the gradient of the loss function with respect to parameters in a neural network.



Reading

Reading:

Deep Learning, 6.5-6.6





Backpropagation

Intuitively, backpropagation is the application of the chain rule for derivatives.

- Repeated application of chain rule of calculus
- Computationally efficient manner given a particular computation graph

Motivation for backpropagation

To do gradient descent, we need to calculate the gradient of the objective with respect to the parameters, θ. However, in a neural network, some parameters are not directly connected to the output. How do we calculate then the gradient of the objective with respect to these parameters? Backpropagation answers this question, and is a general application of the chain rule for derivatives.



Backpropagation

Nomenclature

Forward propagation:

- Forward propagation is the act of calculating the values of the hidden and output units of the neural network given an input.
- It involves taking input $\mathbf{x}$, propagating it to through each hidden unit sequentially, until you arrive at the output $\mathbf{y}$. From forward propagation, we can also calculate the cost function $J(\theta)$.
- In this manner, the forward propagated signals are the activations.
- With the input as the "start" and the output as the "end," information propagates in a forward manner.

Backpropagation (colloquially called backprop):

- As its name suggests, now information is passed backward through the network, from the cost function and outputs to the inputs.
- The signal that is backpropagated are the gradients.
- It enables the calculation of gradients at every stage going back to the input layer.



Why do we need backpropagation?

- Backpropagation is computationally efficient because we will find that most of the terms used in backpropagation can be cached from the forward pass.
- Informally (at least for me) sometimes taking analytical multivariate gradients can be challenging. Backpropagation breaks down these multivariate gradients into easier steps.
- It "operationalizes" gradients, giving a simple algorithm to compute gradients in a neural network. This is critical for neural network libraries that include auto-differentiation.

A few further notes on backpropagation.

- Backpropagation is not the learning algorithm. It's the method of computing gradients.
- Backpropagation is not specific to multilayer neural networks, but a general way to compute derivatives of functions.

A simple example

$$f(x, y, z) = (x + y)z$$

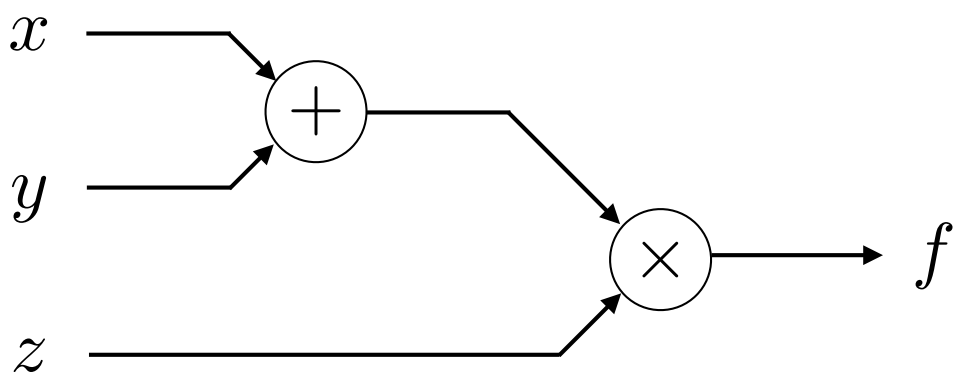
$\rightarrow xz + yz$

GOAL: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$\frac{\partial f}{\partial x} = z$

$\frac{\partial f}{\partial y} = z$

$\frac{\partial f}{\partial z} = (x+y)$

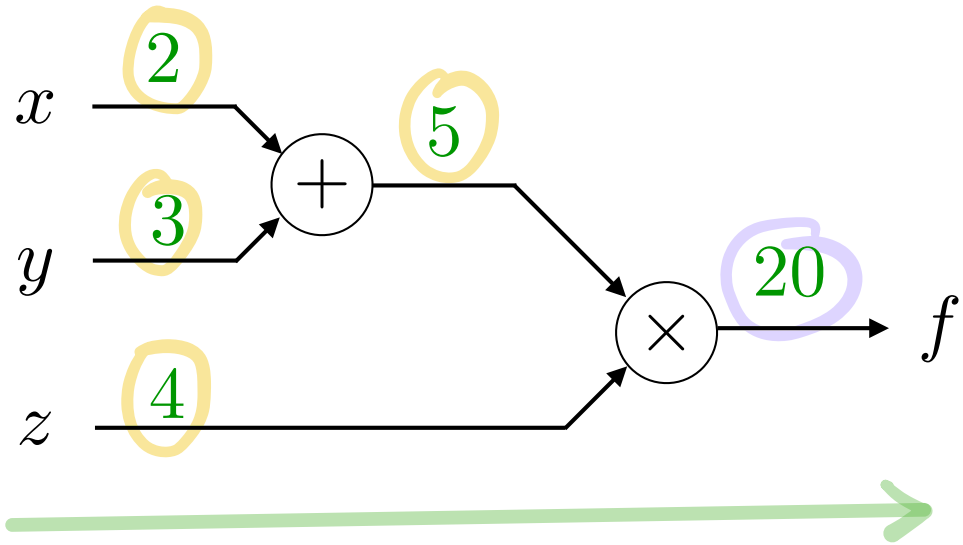




A simple example

$$f(x, y, z) = (x + y)z$$

Forward propagation:



A simple example

$$f(x, y, z) = (x + y)z$$

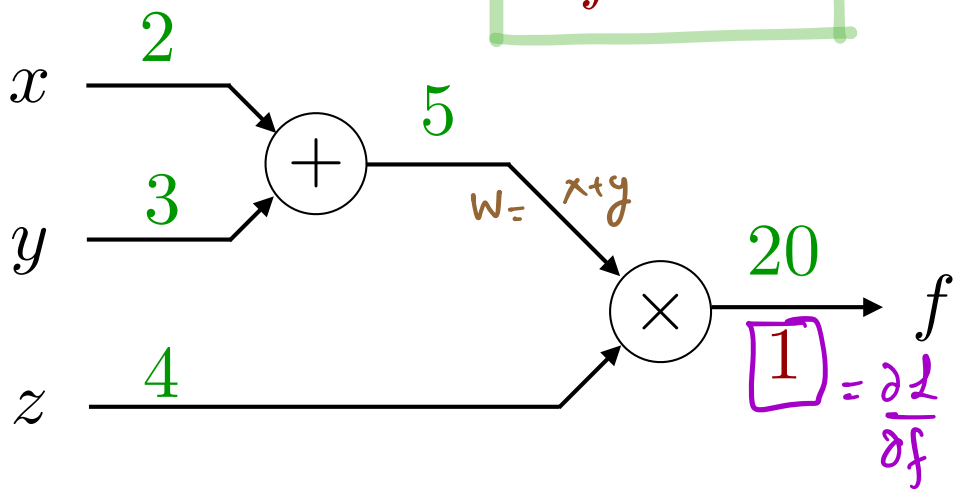
upstream gradients are known

Backpropagation:

$$\frac{\partial \mathcal{L}}{\partial f} = 1$$

given

$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$\frac{\partial \mathcal{L}}{\partial f}$

$f = w \cdot z$

$\frac{\partial f}{\partial z} = w$

Chain rule

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial f}{\partial z} \cdot \frac{\partial \mathcal{L}}{\partial f}$$

$\frac{\partial \mathcal{L}}{\partial f} \to 1$

local gradient

A simple example

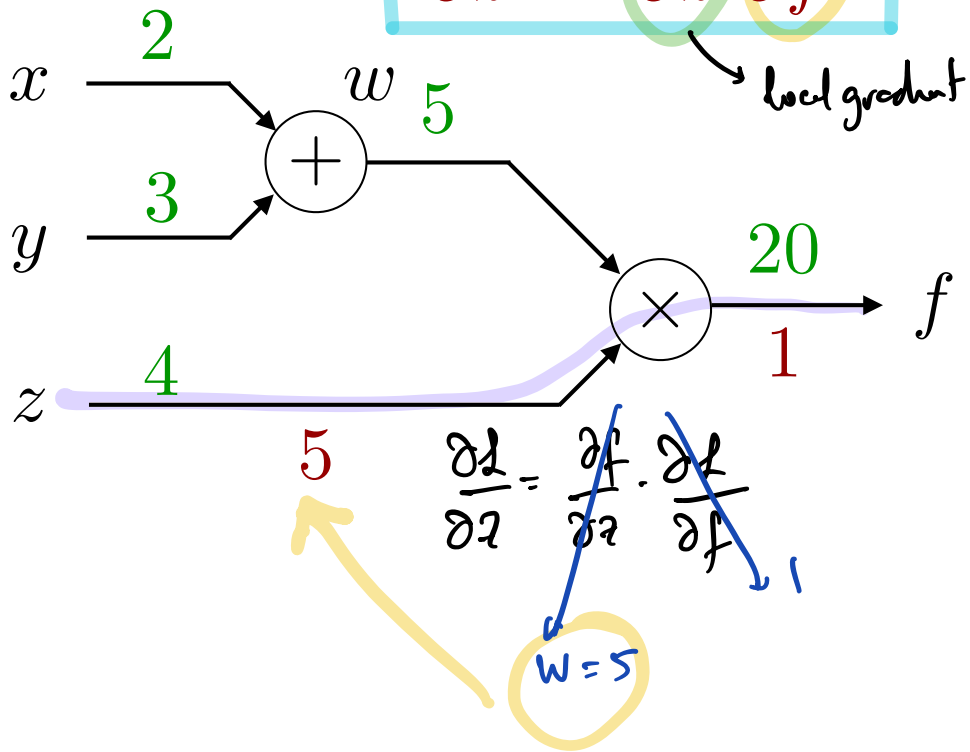
$$f(x, y, z) = (x + y)z$$

Backpropagation:

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial f}{\partial z}\frac{\partial \mathcal{L}}{\partial f}$$

$\frac{\partial \mathcal{L}}{\partial f} = 1$ (upstream gradient given)

local gradient



A simple example

$$f(x, y, z) = (x + y)z$$

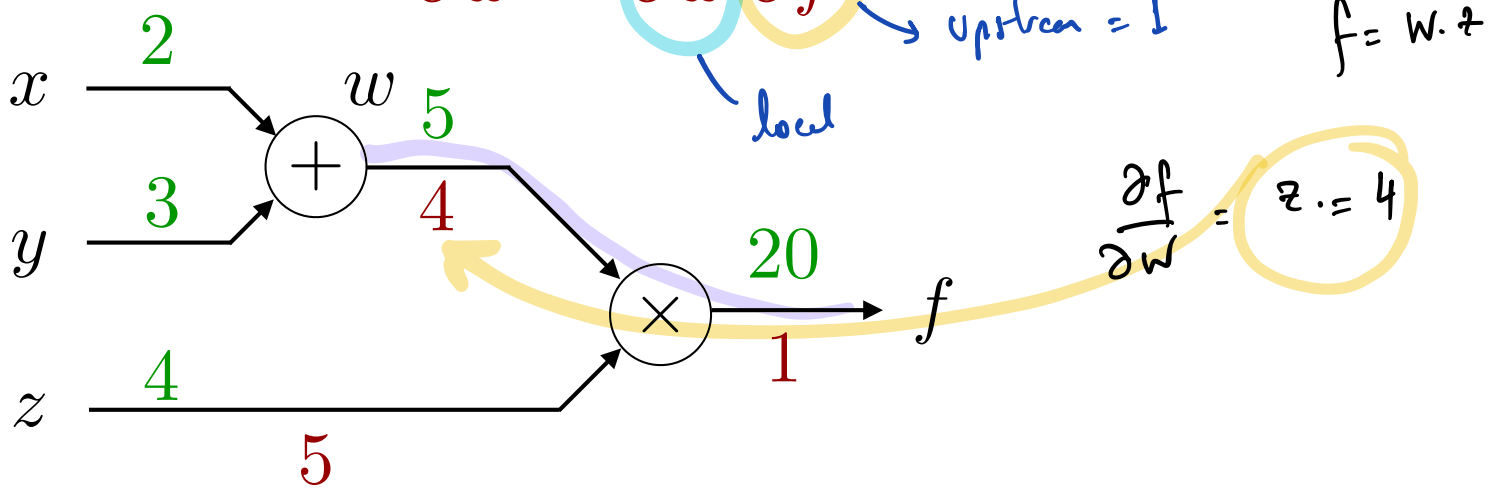
Backpropagation:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial f}{\partial w}\frac{\partial \mathcal{L}}{\partial f}$$

upstream = 1

local

f = w.r.t



A simple example

$$f(x, y, z) = (x + y)z$$

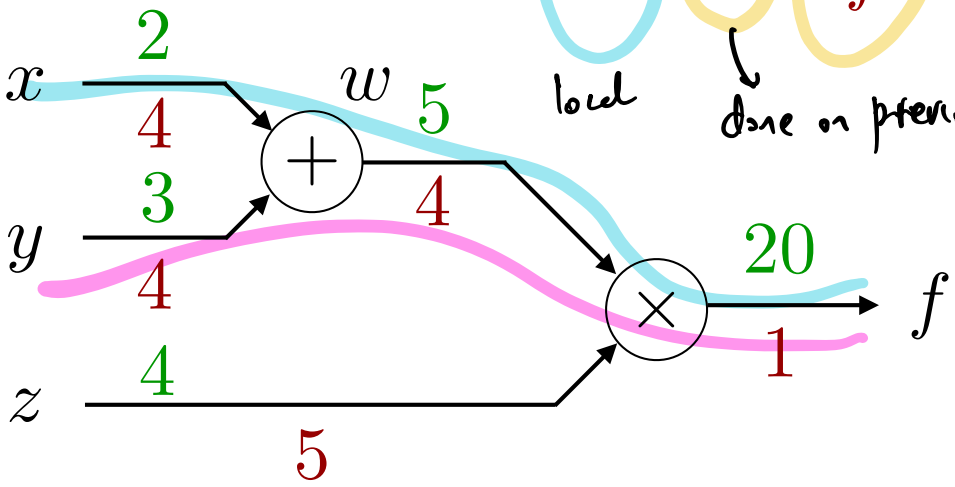
Backpropagation:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial w}{\partial x}\frac{\partial f}{\partial w}\frac{\partial \mathcal{L}}{\partial f}$$

local

done on previous slide

known from initial

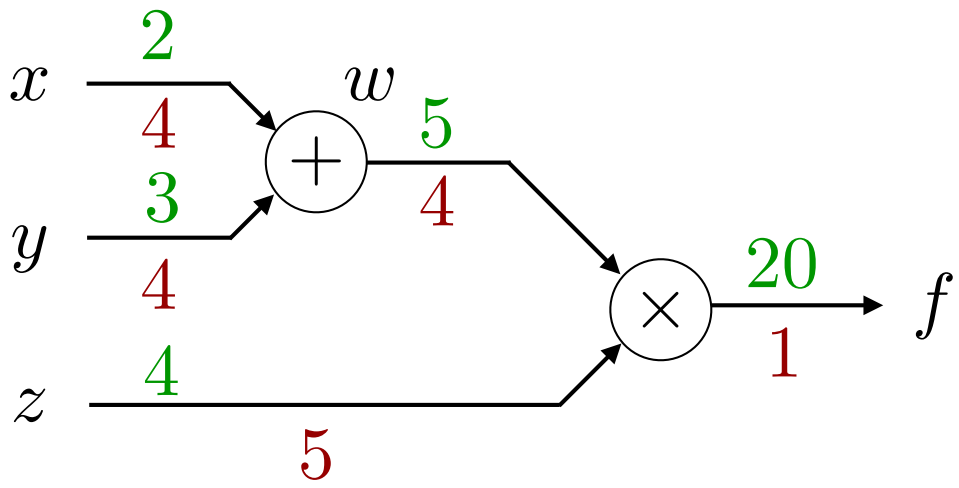


$w = x + y$

$$\frac{\partial w}{\partial x} = 1 = \frac{\partial w}{\partial y}$$

Idea: computational graphs apply to gradients

In the **forward pass**, we apply a function to the node inputs to calculate an output.
In the **backward pass**, we take the **upstream derivative** and apply a **local gradient** to calculate the backpropagated derivative.

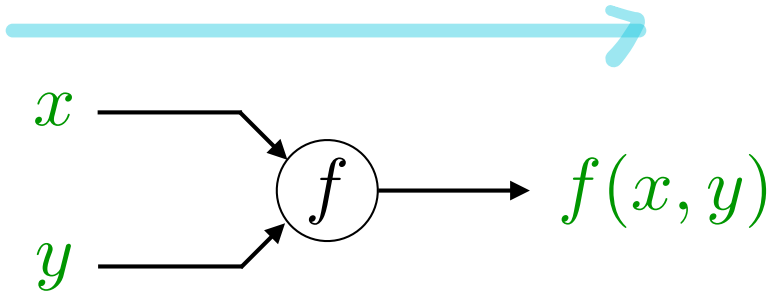




Idea: computational graphs apply to gradients

In the **forward pass**, we apply a function to the node inputs to calculate an output.
In the **backward pass**, we take the **upstream derivative** and apply a **local gradient** to calculate the backpropagated derivative.

Forward pass:

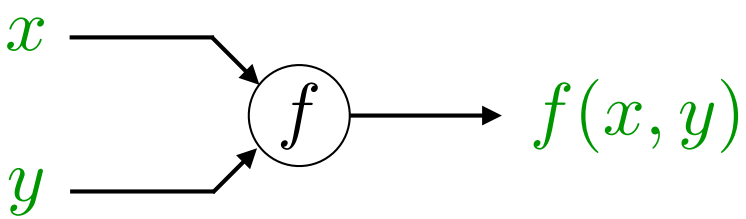




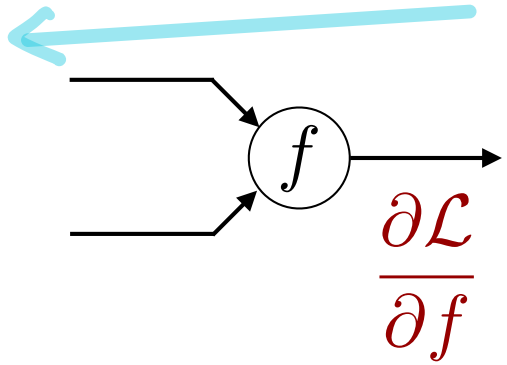
Idea: computational graphs apply to gradients

In the **forward pass**, we apply a function to the node inputs to calculate an output.
In the **backward pass**, we take the **upstream derivative** and apply a **local gradient** to calculate the backpropagated derivative.

Forward pass:



Backward pass:

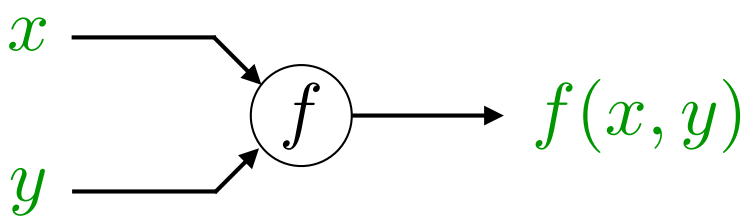




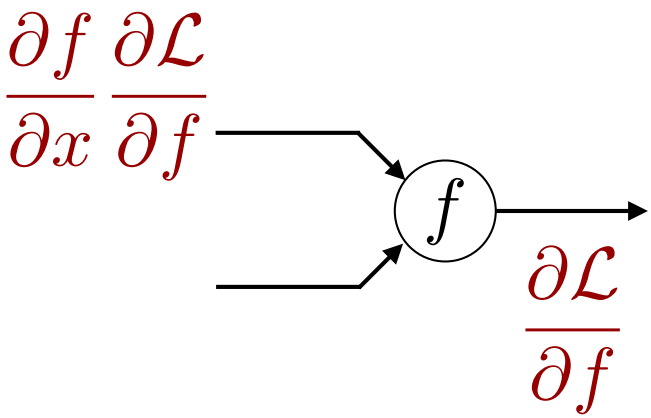
Idea: computational graphs apply to gradients

In the **forward pass**, we apply a function to the node inputs to calculate an output.
In the **backward pass**, we take the **upstream derivative** and apply a **local gradient** to calculate the backpropagated derivative.

Forward pass:



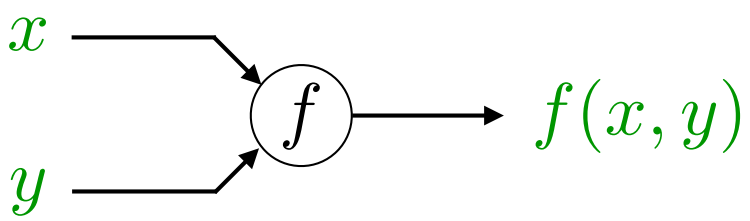
Backward pass:



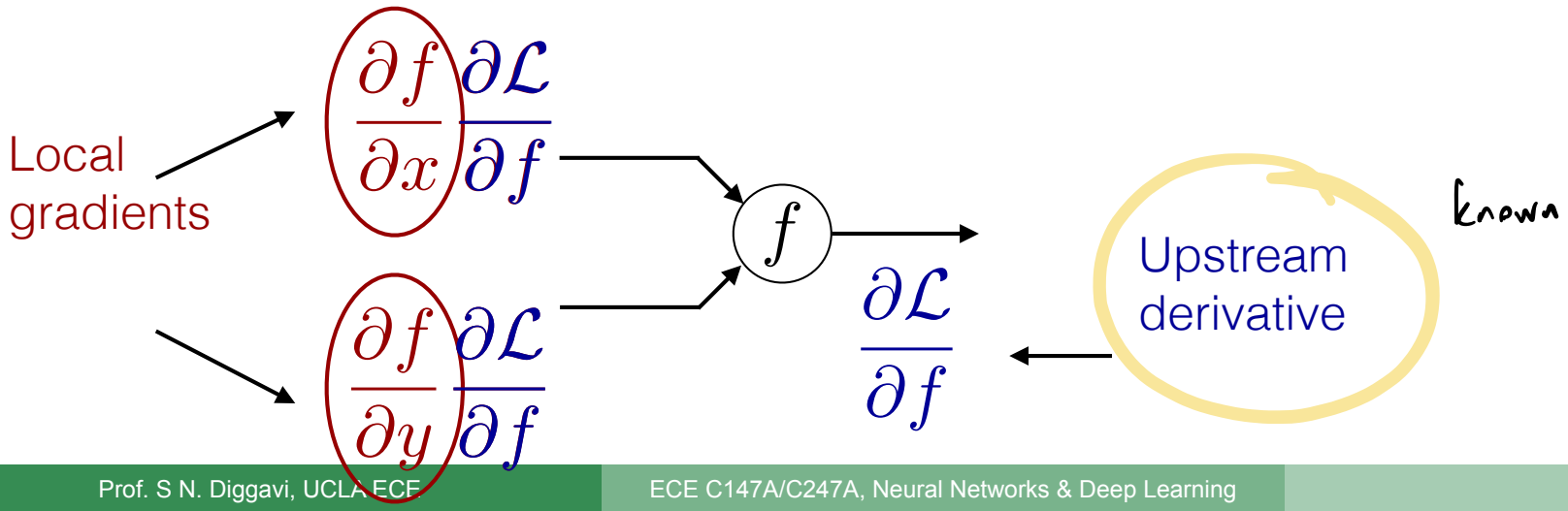
Idea: computational graphs apply to gradients

In the **forward pass**, we apply a function to the node inputs to calculate an output.
In the **backward pass**, we take the **upstream derivative** and apply a **local gradient** to calculate the backpropagated derivative.

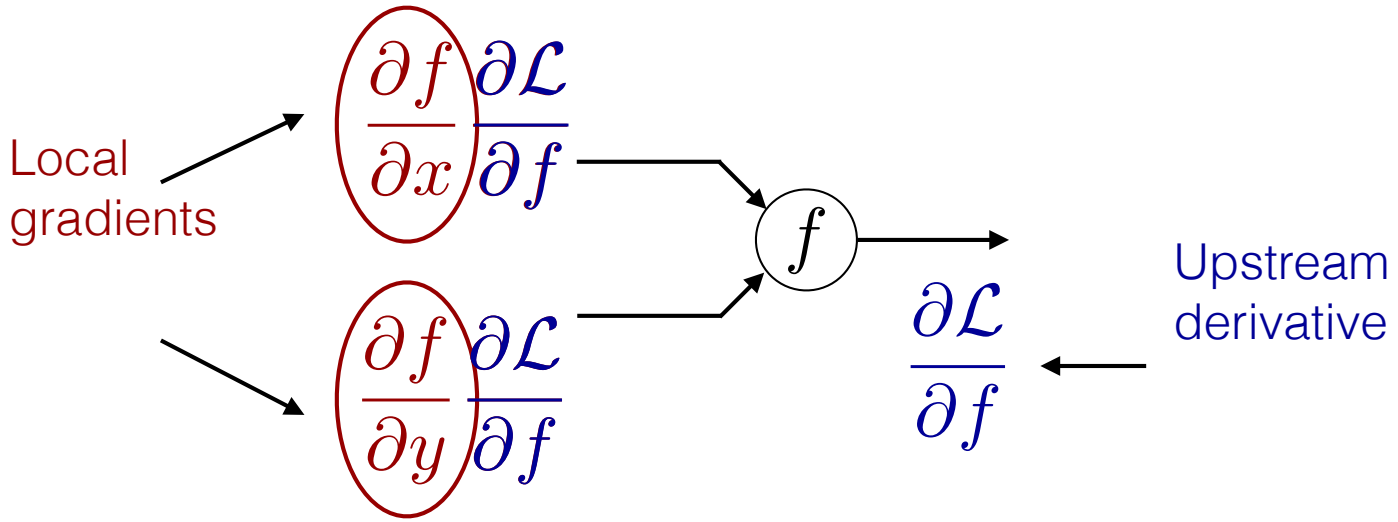
Forward pass:



Backward pass:



Idea: computational graphs apply to gradients



The basic intuition of backpropagation is that we break up the calculation of the gradient into **small and simple steps**. Each of these nodes in the graph is a straightforward gradient calculation, where we multiply an **input** (the "upstream derivative") with a **local gradient** (an application of the chain rule).

Composing all of these gradients together returns the overall gradient.

A gate view of gradients

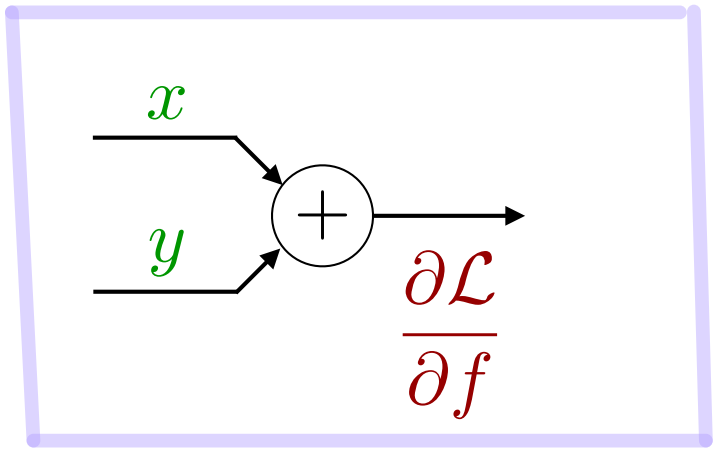
Interpreting backpropagation as gradient “gates”:

$$f = x + y$$

$$\frac{\partial f}{\partial x} = 1 = \frac{\partial f}{\partial y}$$

$$\frac{\partial L}{\partial x} = \frac{\partial f}{\partial x} \cdot \frac{\partial L}{\partial f}$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial f}$$





A gate view of gradients

Interpreting backpropagation as gradient “gates”:

Add gate: distributes the gradient

